
就職活動用 プログラム説明資料

『Irufemi Engine』

エディタ & AIシェーダー生成ツ
ール
機能紹介



使いやすさを追求したエディタ&デバッグ環境

<開発の目的>

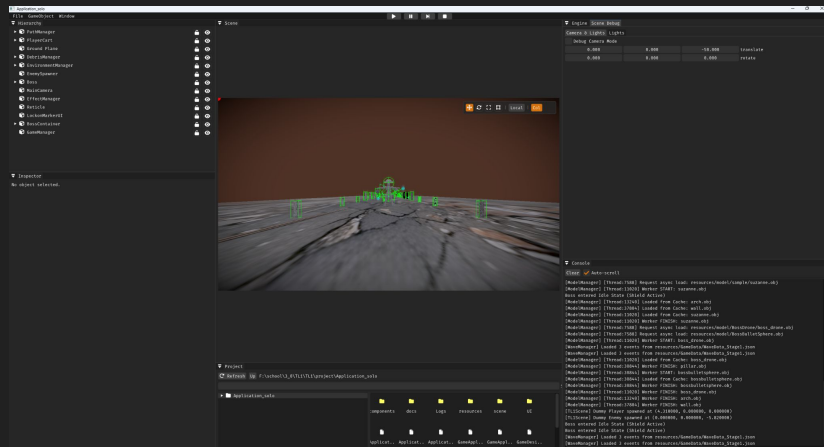
パラメータ調整のたびに発生する「再ビルドの待ち時間」や、Releaseビルドでの「正確な負荷が分からない」といった開発中の課題を解消し、イテレーションを高速化することを目的として構築しました。

■ イテレーションの効率化

ゲームを再起動せず、その場で全てを調整・確認できるホットリロード設計

■ 本番環境での計測

ゲーム本体に負荷をかけずにプロファイリングできる外部監視システム



コードをいじらずに「即配置・即テスト」できるレベルエディタ

ローダーと配置、ホットリロード、無効フラグ

<課題>

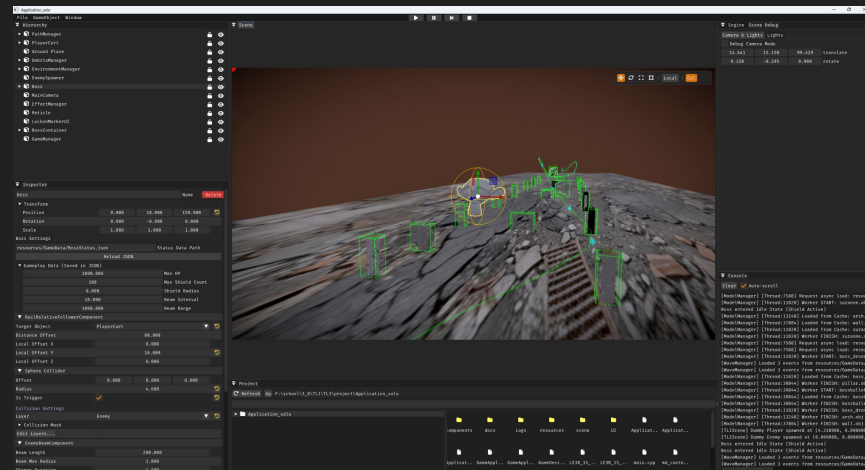
敵の配置や当たり判定の微調整のために、いちいちソースコードの数値を書き換えて再コンパイルするのは非常に非効率でした。

<解決策>

エディタ上で自作モデルやコライダー、イベントトリガーを直感的に配置・保存できる仕組みを構築し、それをゲーム側でそのまま読み込めるようにしました。

<結果>

手を煩わせることなく、エディタ上だけでレベルデザインとゲームプレイ of テストが完結するようになりました。



変数登録だけで UI化、そして再起動不要の「ホットリロード」

<課題>

エディタにパラメータ UIを追加するたびに、GUI用のコードを書くのが手間で、保守性も悪化していました。

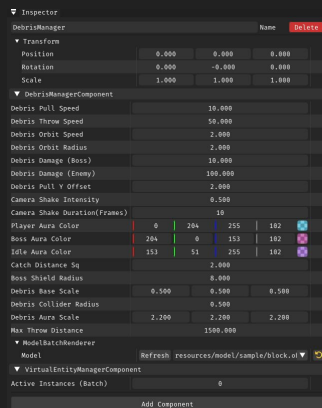
<解決策>

C++側で変数を登録するだけでインスペクター UIが自動生成される「リフレクション基盤」を独自に実装。さらに、ゲーム側からリロードボタンを押すだけで最新のレベルデータを再読み込みする機能を作りました。

<結果>

GUIの追加工数がゼロになり、ゲームを実行したままパラメータを調整して即座に反映できる、快適なイテレーション環境を手に入れました。

```
void DebrisManagerComponent::OnRegisterProperties() {  
    Component::OnRegisterProperties();  
    RegisterProperty("Debris Pull Speed", &debrisPullSpeed_);  
    RegisterProperty("Debris Throw Speed", &debrisThrowSpeed_);  
    RegisterProperty("Debris Orbit Speed", &debrisOrbitSpeed_);  
    RegisterProperty("Debris Orbit Radius", &debrisOrbitRadius_);  
    RegisterProperty("Debris Damage (Boss)", &debrisDamage_);  
    RegisterProperty("Debris Damage (Enemy)", &debrisEnemyDamage_);  
    RegisterProperty("Debris Pull Y Offset", &debrisPullYOffset_);  
    RegisterProperty("Camera Shake Intensity", &cameraShakeIntensity_);  
    RegisterProperty("Camera Shake Duration(Frames)", &cameraShakeDurationFrames_);  
    RegisterProperty("Player Aura Color", &playerAuraColor_);  
    RegisterProperty("Boss Aura Color", &bossAuraColor_);  
    RegisterProperty("Idle Aura Color", &idleAuraColor_);  
    RegisterProperty("Catch Distance Sq", &catchDistanceSq_);  
    RegisterProperty("Boss Shield Radius", &bossShieldRadius_);  
  
    RegisterProperty("Debris Base Scale", &debrisBaseScale_);  
    RegisterProperty("Debris Collider Radius", &colliderRadius_);  
    RegisterProperty("Debris Aura Scale", &auraScale_);  
    RegisterProperty("Max Throw Distance", &maxThrowDistance_);  
}
```



開発初期の「仮シェーダー」を AIで瞬時に構築する「AIMagicBrush」

テキストと参考画像を指定するだけで、AIがHLSLシェーダーコードを自動生成して出力します。

<課題>

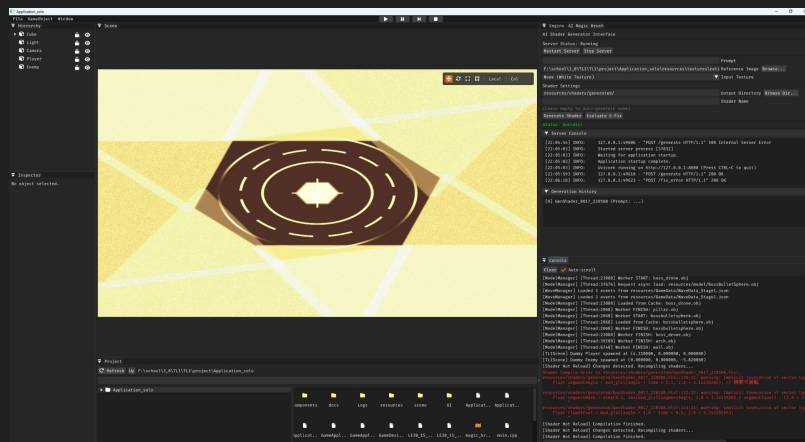
本番用のアセットができるまでのプロトタイプ段階において、少し質感を付けたい場合でも、自分で「仮のシェーダーコード」を一から組むのは時間と手間がかかるという課題がありました

<解決策>

生成AIをツールに組み込み、PythonのAIサーバーと非同期通信を行うアーキテクチャを採用しました。コンパイルエラーが出てもAI自身がログを解析して自動修復(自己修復ループ)します。

<結果>

ざっくりとした指示を出すだけで、ゲームの進行を一切ブロックすることなくバックグラウンドで「仮シェーダーのベースコード」が生成され、手軽にプロトタイピングの準備ができるようになりました。



Release環境の負荷を正確に計測する外部プロファイラ

ゲームとは別プロセスで動作するダッシュボードUDP通信により、ゲーム画面を塞ぐことなくリアルタイムFPSやスレッド稼働状況を監視できます。

<課題>

エンジン内蔵のImGuiプロファイラは、Releaseビルド時に無効化されたり、描画そのものに負荷がかかるため、「本番環境での正確なパフォーマンス計測」ができない不満がありました。

<解決策>

AAAタイトルの手法を参考に、UDPでシンプルなJSONデータを送信し、別プロセス(C# WPFアプリ)で受信・グラフ化する「疎結合なアーキテクチャ」を構築しました。

<結果>

ゲーム本体に一切のオーバーヘッドを与えずにプロファイリングが可能に。さらに自作エンジンに限らず、どんなプロジェクトにも明日から導入できる高い汎用性を実現しました。

